FlagPrism: Compiler-Assisted Fine-Grained Profiling Across AI Accelerators (Operational Systems)

Anonymous Authors

Abstract

[TODO: Write the abstract after confirming the study findings, system mechanisms, verified hardware coverage, and operational experience. State measured results only after experiments; the working title is provisional.**]**

1 Introduction

Unified kernel programming frameworks allow operator developers to reuse program logic across artificial intelligence (AI) accelerators. For example, FlagTree extends the Triton ecosystem to multiple accelerator backends [1]. Obtaining high performance still requires understanding how a target executes that logic: where computation, data movement, and synchronization consume time; whether these activities overlap; and how tiling, layout, and pipeline choices change execution. These profiling questions recur across devices, even when their answers require different execution objects and measurement mechanisms. A useful profiler must connect the resulting observations to the operator implementation being optimized.

Architecture diversity appears at more than one level. Graphics processing units (GPUs) with similar single-instruction, multiple-thread (SIMT) execution models can differ in thread organization, local-memory structure, resource limits, and asynchronous facilities [7, 8]. Accelerators with dedicated computation and data-movement pipelines introduce further differences in how operations are scheduled and synchronized [4]. Such differences affect profiling as well as optimization. A source region can be decomposed, fused, reordered, or pipelined during compilation; timing the issue of an asynchronous operation need not measure its execution or completion. A common interface or output format therefore does not, by itself, determine which execution events should be observed, where instrumentation belongs, or how records should be related back to a program region.

Existing work supplies important foundations. KPerfIR integrates fine-grained profiling into compiler intermediate representation (IR) and analyzes intra-kernel execution and overlap [2]; Neutrino supports programmable instruction-level probes [3]. Proton combines vendor measurement interfaces and instrumentation with multi-level program context [10], while PASTA provides a modular analysis framework for accelerator workloads [6]. These capabilities motivate a concrete question for a broader collection of execution backends: which program mappings, instrumentation semantics, and measurement assumptions can be reused, and which must change when the target architecture changes? **[TODO:** Verify the exact supported versions and mechanisms of each baseline, and replace this scoped question with evidence-supported limitations from the study. Avoid claiming unsupported platforms are inherently impossible to support.**]**

This paper investigates the portability of *fine-grained profiling intent*: a developer's request to observe a logical operator region or an execution behavior relevant to optimization. We separate three difficulties. First, compilation changes the relationship between logical regions and execution instances. Second, instrumentation must respect the target's execution semantics, including the distinction between submission, execution, and waiting. Third, timestamps, recording mechanisms, and vendor observations must be reconstructed into program-associated results without hiding architecture-specific detail. These difficulties occur both within a broadly shared execution model and across substantially different execution models.

We place an empirical study before the system design. Six operators on NVIDIA V100S, Enflame GCU300, and Ascend 910B4-1 establish the range of behavior; fused GEMM and softmax then carry the same body/output observation request through their compiled execution objects. The experiments distinguish correct operator outputs from valid profiling scope, actor identity, and counter protocols. Compiler-assisted actor records and a fixed-work dependency control check the resulting requirements. Differences in device speed alone do not establish a profiling limitation. Results concern these backend/software-stack combinations; they do not establish a same-SIMT-model comparison across vendors.

We investigate FlagPrism, a profiler whose system boundary includes the profiling-related compilation mechanisms in FlagTree, together with runtime collection, event association, and analysis. Its intended service is to lower common profiling requests into target-appropriate observations and reconnect those observations to the operator program. It provides profiling services to operator developers and optimization tools. The compiler–profiler boundary and its concrete mechanisms will be defined from the study rather than fixed in advance. **[TODO:** Finalize whether profiling intent requires a new dialect, extensions to existing IR, metadata and passes, or a combination. Distinguish inherited compiler/profiler functionality, implemented extensions, and proposed work.**]**

The planned contributions are:

- An empirical characterization of fine-grained profiling portability across three accelerator backend stacks. Six-operator coverage and two real-operator cases establish the scoped mapping, boundary and clock-protocol findings in Section 3.
- Compiler–profiler mechanisms for program mapping, target-specific instrumentation, and program-associated reconstruction. **[TODO:** Name and substantiate only the mechanisms actually implemented.**]**
- A system evaluation and operational account of profiling-guided operator optimization, including measurement fidelity, perturbation, unsupported observations, and unsuccessful diagnoses. **[TODO:** Supply actual case studies and usage records; no deployment or performance claim is established yet.**]**

2 Background and Profiling Scope

2.1 Operators, Backends, and Execution Models

[TODO: Define the operator programming model, supported input/configuration space, backend compiler versions, and concrete chip models. Separate vendor, architecture generation, execution model, and software stack. Describe NVIDIA, Iluvatar, and Ascend at verified granularity for the study; introduce additional vendors only as needed for system evaluation.**]**

2.2 From Program Regions to Device Execution

A logical region is a developer-visible part of an operator. An execution instance is the concrete work associated with a particular invocation and configuration. Their correspondence may be one-to-many or many-to-one. **[TODO:** Choose the exact identity and provenance model after inspecting backend IR; define treatment of fusion, duplication, loops, and overlapping regions.**]**

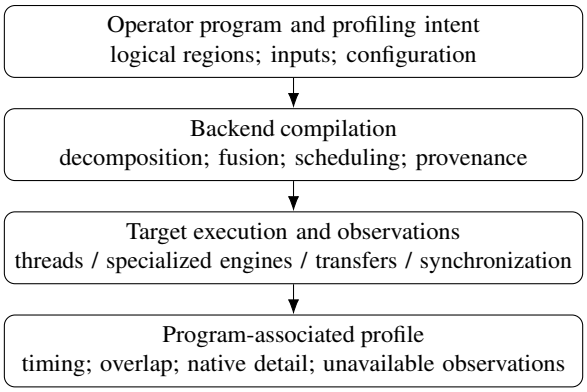


Figure 1: Conceptual profiling scope. The branches illustrate possible mappings, not measured schedules or a claim about a particular chip. The study will replace them with compiler-derived and runtime-validated examples.

2.3 Measurement Semantics

[TODO: Define host submission, device execution, asynchronous completion, waiting, overlap, timestamp domains, and estimated versus measured quantities. State which quantities cannot be observed on each tested environment.**]**

3 Study on Cross-Architecture Profiling

Logic. Program/execution differences → consequences for the same profiling request → controlled checks → compiler–profiler requirements.
Scope. NVIDIA V100S, Enflame GCU300, Ascend 910B4-1; six fixed-source probes; three shapes and two configurations per probe. Byte-identical inputs → CPU-reference checks → saved compiler artifacts → separate baseline/native captures. All 216 cases pass; 1,620 native intervals and 1,296 artifact hashes are audited. These are backend-stack observations, with neither fixed clocks nor exclusive hardware. The sample ranges are descriptive, not confidence intervals.
Fixed observation request. For each logical program instance, report the *body* region (before input loads through the selected post-output-store barrier) and the *output* region (downstream entry through that barrier). Output includes readiness wait and workspace/input transfer executed within its boundary. Preserve actor identity, program coordinates, invocation/capture identity, endpoints, clock domain, units, and included activities. A region split across actors is represented by actor projections; an overall interval remains unavailable until a clock relation is verified. Separate replay captures satisfy separate observations, not a joint timeline. For the fused

Ascend projection, AIC body ends after its workspace-output completion barrier; AIC output is not applicable. AIV body ends after final output completion and its output starts before readiness wait. For softmax, output starts after the producer sum is materialized, before divide/store. GPU records use the designated recording work-item (thread0 on V100, work-item0 on GCU), ending at the selected group barrier. They are control-path projections, not minimum-entry/maximum-exit envelopes over all lanes. The same request is applied below to fused GEMM and softmax. Numerical output correctness does not, by itself, establish observation identity or timing scope.

3.1 One Operator Group, Two Execution Cases

Breadth. Six operators $\rightarrow$ three shapes and two configurations $\rightarrow$ duration distributions and configuration responses. Performance differences motivate inspection; they do not prove a profiling defect.

Compiled intervention. GEMM stages1/2 $\rightarrow$ identical GCU/V100 binaries at all three shapes; changed Ascend buffering/events. Same parameter name $\neq$ equivalent compiled change.

Case 1: fused GEMM+bias+GELU (128^3). One source program $\rightarrow$ one V100 PTX entry / one GCU kernel / Ascend AIC plus two AIV subinstances per program $\rightarrow$ workspace and cross-core set/wait dependencies. Aggregate native activity: one kernel/task interval per launch.

Case 2: softmax (32×256). Load $\rightarrow$ max/exp/sum $\rightarrow$ divide/store; V100 thread/warp reductions, GCU local reductions, Ascend single-AIV projection $\rightarrow$ different synchronization and observation protocols without the fused case's AIC/AIV decomposition.

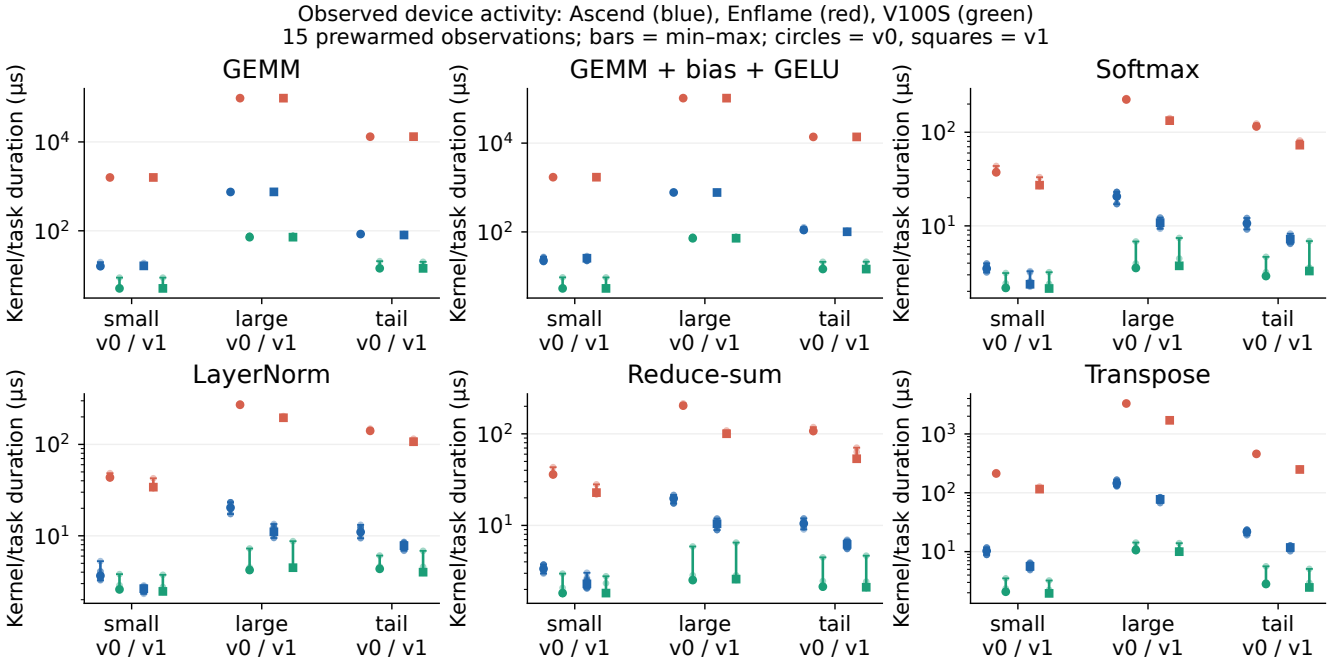


Figure 2: Six untuned probes, 15 prewarmed native intervals per configuration. Small dots: observations; larger circle/square: v0/v1 medians; bars: min–max. GEMM shapes: 128^3, 512^3, $257 \times 193 \times 320$; row shapes: 32×256, 256×1024, 127×769; transpose shapes: 256^2, 1024^2, 513×257. v0/v1 request stages1/2 (GEMM), row grouping1/2, or transpose tile M16/32. Compiler stacks differ; no fixed-clock or exclusive-device guarantee.

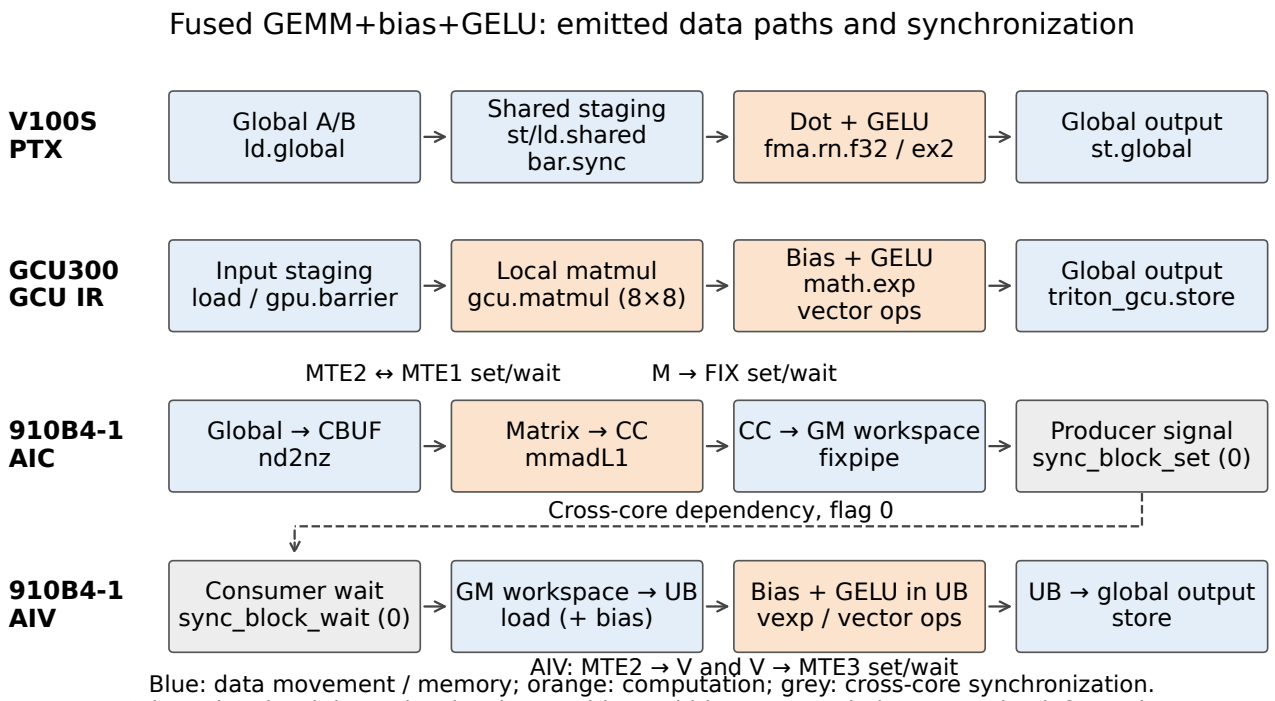


Figure 3: Selected emitted fused-kernel data paths and synchronization. Arrows are dependencies, not a time axis. Ascend splits matrix production and output processing across AIC/AIV functions; each AIC program has two AIV subinstances in the studied configuration. The diagram does not infer overlap.

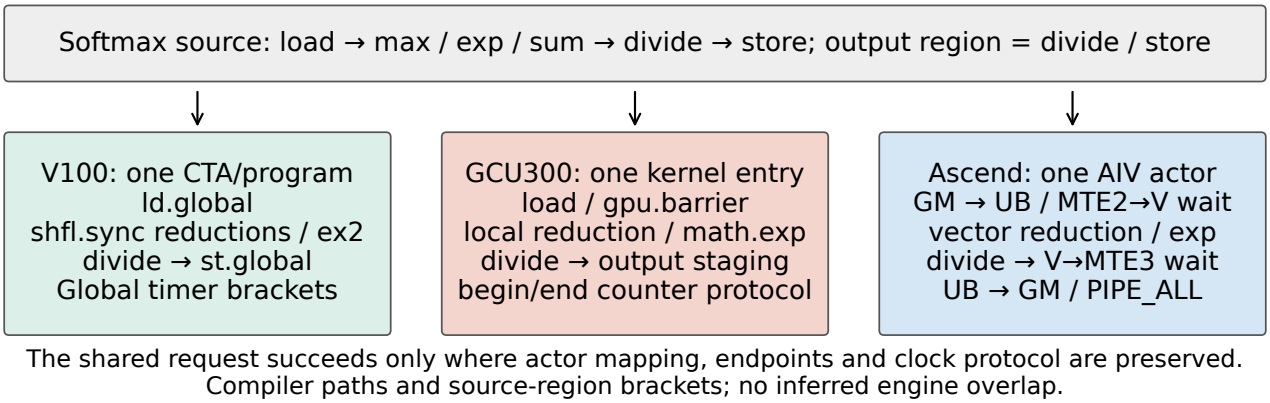


Figure 4: Actual softmax lowering paths. The output region covers final normalization and store. One AIV actor per row program makes Ascend source-level body/output recording a successful contrast to its fused case.

3.2 The Same Request Exposes Concrete Consequences

Clock protocol, both operators. Generic nested begin/end subtraction → GCU body and output end values become identical; both starts are zero → body coverage is lost even though operator outputs remain correct. Separate single-active captures → recover distinct requested windows. A research LLVM adapter selects the installed SDK read-only timer → one invocation retains ordered body/output timepoints and work-item0 records. V100 uses its existing stateless timer. At tile16, GCU window-counter body is 2.25% of its single-active control; softmax rows1 is 31.66%. A one-program isolation control reproduces both effects. The timer adapter preserves nesting, but extra timer reads perturb softmax: its nested/single-active body ratio is 1.239, versus 1.008 for fused GEMM. These are protocol and perturbation observations, not timing-accuracy errors.

Mapping and scope, fused Ascend. Source-level recording passes correctness and program-ID checks → AIC observations are not returned; both AIV subinstances target the same program-only record slot → returned actor provenance is unidentifiable. Its output clock starts after the original wait and workspace load issuance → it excludes readiness wait from the requested scope. These are emitted-code facts, not measured mixed-field or mismatch rates.

Backend adaptation. Preserved launch ABI + pre-link LLVM instrumentation → explicit (program, AIC/AIV, sub-block, invocation) slots → 64 AIC and 128 AIV records per fused launch; output entry before the AIV wait → directly recorded wait. Softmax source recording already preserves its single-AIV body/output request; late instrumentation supplies additional observation boundaries. These are research probes, not a claim of completed production backend support.

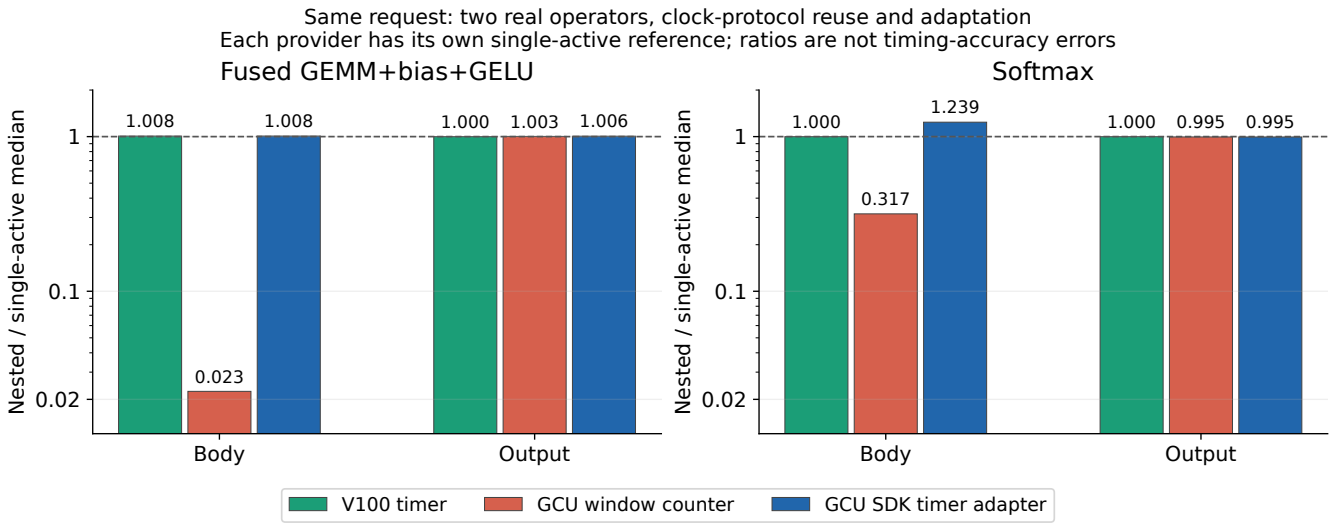


Figure 5: The same body/output request on two actual operators. Ratios compare pooled medians from 15 launches per setting; the logarithmic axis shows protocol consistency, not physical-time accuracy. Each provider uses its own single-active control in separate captures. The GCU adapter changes the primitive to an SDK timer; its matched controls use that timer too. Reference ratios do not compare different native units. GCU replay frames are different physical invocations; only the stateless adapter provides both regions from one invocation. The baseline is a generic nested subtraction probe, not a reproduction of Proton/KPerfIR.

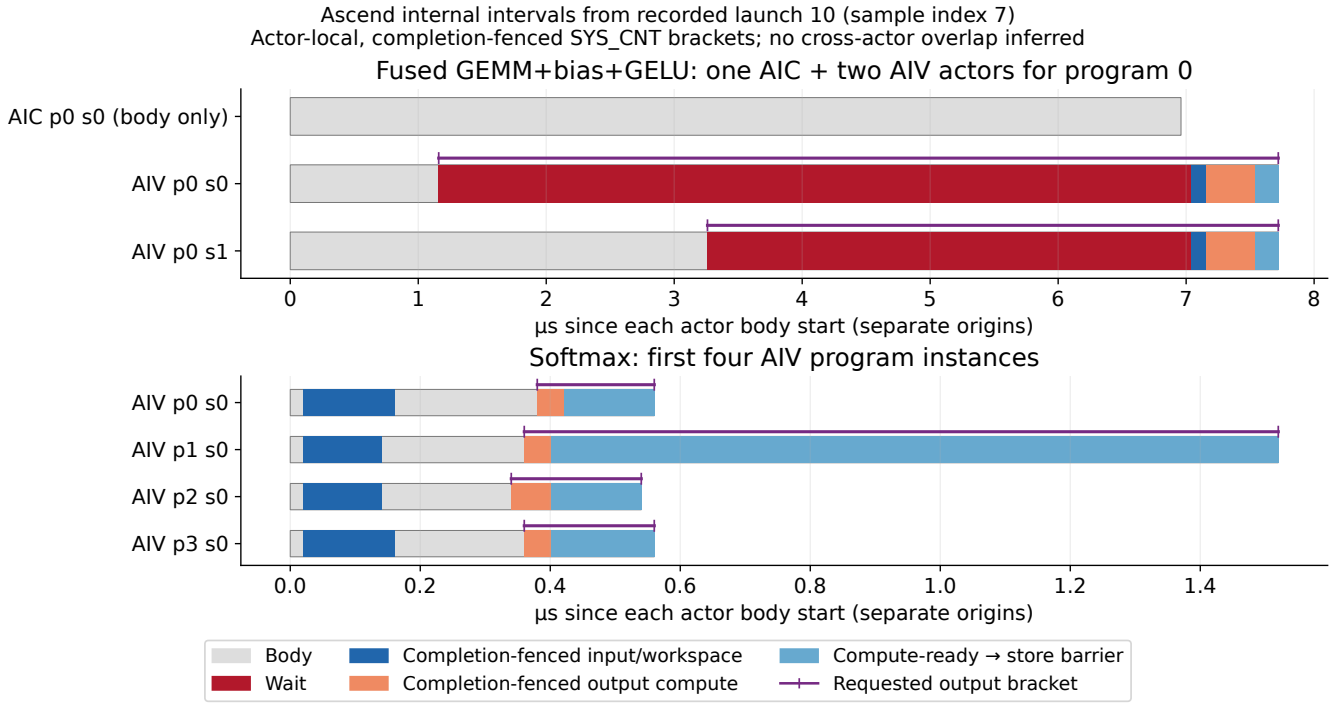


Figure 6: Recorded Ascend internal brackets for both real operators, sample index7 (invocation10 after three warmups). Each row has its own body-start origin; rows do not establish cross-actor overlap. SYS_CNT uses 50 ticks/μs, verified against the installed SDK implementation and its documented convention [5]. The grey AIC row reports only its body; AIC phase decomposition is unavailable. Shown AIV phase endpoints add explicit PIPE_ALL completion fences; the intervals include their perturbation and clock-read costs. Zero-tick segments are unresolved observations, not zero-cost operations.

3.3 Controlled Verification

Dependency control. Fixed inputs/math + 16,384 total auxiliary NOPs + identical producer completion fence → move 0/8,192/16,384 NOPs before the original signal → unchanged AIV code and wait endpoints → wait medians 222/367.5/1,036 ticks (4.44/7.35/20.72μs). The first unfenced control did not separate delay from outstanding producer work; it is retained as an unsuccessful control. Equal instruction count does not imply equal elapsed time.

Endpoint control, both cases. An observer on PIPE_S → a wait/gate between other pipelines does not establish its completion boundary. Keep math, inputs and actor layout; add explicit PIPE_ALL completion endpoints → fused workspace 0→6 ticks, epilogue 5→19, suffix-to-store 26.5→9. Softmax shows the corresponding input/normalize changes. The original brackets are retained as issue/gate observations; the new fences change execution, so their differences are not an accuracy-error estimate.

Identity and numerical checks. Byte-identical inputs and CPU outputs → unique expected actor slots → serial launch/native-record association. V100: 6,960 recorded region durations fit within their independently collected same-launch CUPTI whole-kernel durations. Ascend: actor projections fit within same-launch aclprof task durations using the SDK unit conversion. These bound checks do not establish exact internal timing accuracy. An additional V100 probe sandwiches each global-timer endpoint between two `clock64` reads. Across 960 fused and 480 softmax program records, the two counters compare output/body fractions without a frequency conversion. Allowing the observed 32ns timestamp granularity leaves one fused and no softmax non-overlap; that residual is retained. This is a conditional relative check: constant SM frequency within each body is assumed, and the 32ns margin is empirical, not a guaranteed error bound. Absolute internal accuracy remains unverified. V100 recorded timestamps have an observed 32ns quantum. Single-program scale controls use native whole-kernel/task intervals as independent endpoint/collection references. GCU fitting levels 0/64/1024 imply approximately 1ns per SDK timer tick; levels16/256 held out from this fit have native-time residuals $+0.339/-0.245\mu$s. This is a stack-specific empirical calibration, not an official frequency or an internal precision certificate; exploratory all-level data existed before the partition was specified. The documented `clockInstructionRate` property is not bound to this helper and its candidate conversion violates whole-kernel duration bounds. The raw timer unit and unknown official frequency remain explicit.

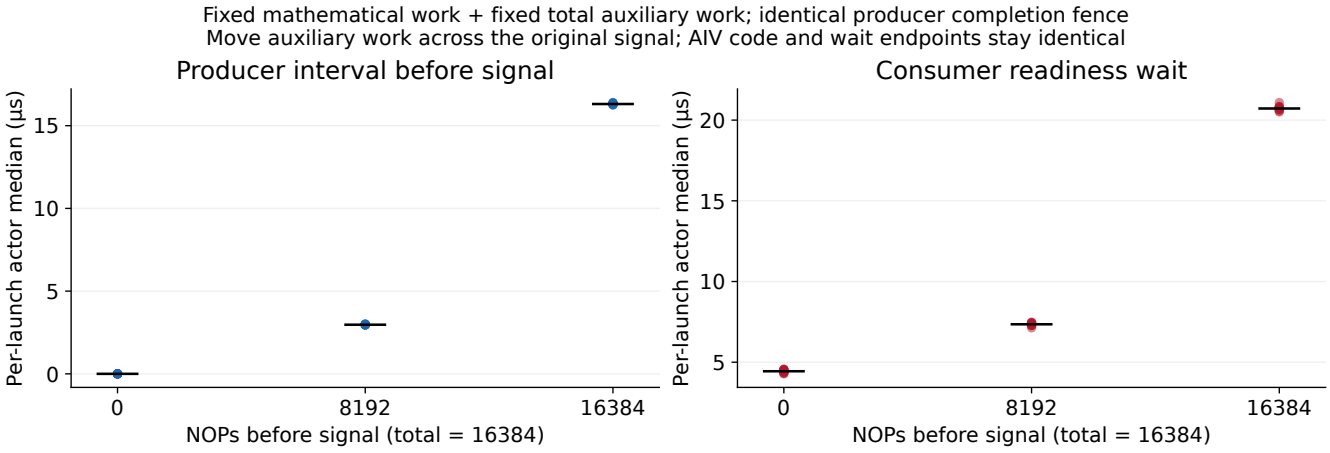


Figure 7: Fused-case dependency intervention with a fixed auxiliary-work budget. Dots are per-launch actor medians (15 launches); black marks are medians of those values. The mathematical output and original signal/wait relation are preserved. Both variants add the same producer completion fence. This is a perturbed readiness control, not useful-compute optimization or unperturbed busy time.

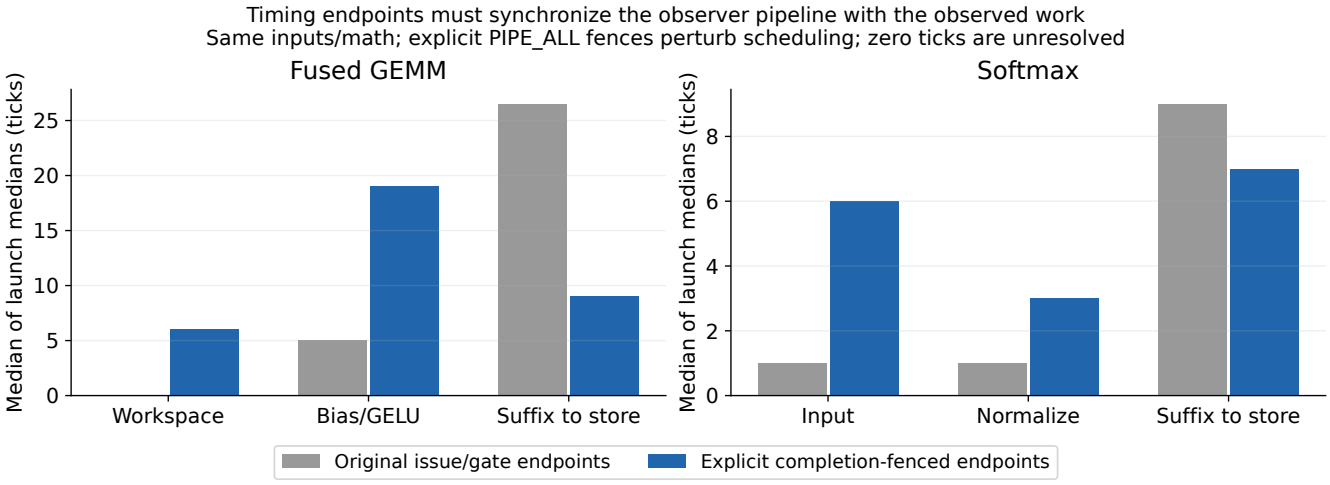


Figure 8: Directly recorded endpoints before and after completion synchronization. Bars are medians of 15 per-launch actor medians. Names identify the intended source component; grey brackets lack completion provenance, while blue brackets include explicit completion fences and their costs.

3.4 Requirements Motivated by the Measurements

- **Compilation provenance and recording identity.** One fused program becomes AIC plus two AIV actors → program-only slots lose AIC records and cannot identify the AIV writer → a source-only recording mechanism cannot preserve the lowered actor mapping. Required mechanism: carry logical-region provenance through lowering and assign actor/subinstance slots. *Implemented:* late-LLVM research instrumentation. *Verified:* 64 AIC and 128 AIV unique slots per studied fused launch, with serial native association. *Proposed:* general compiler-maintained mapping and concurrent collection; an attribution-error rate is not measured.

- **Execution-aware observation boundaries.** The generic fused output clock starts after readiness work → its interval omits requested wait; issue/gate brackets lack completion provenance → source insertion alone cannot define lowered scope. Required mechanism: lower boundaries with dependency, actor and completion semantics. *Implemented:* pre-wait AIV entry and explicit completion probes. *Verified:* unchanged AIV wait code responds to the fixed-work producer intervention; both actual operators retain declared fenced endpoints. *Proposed:* general boundary lowering and perturbation accounting; these probes do not recover unperturbed engine busy time.

- **Clock and collection contracts.** GCU begin/end controls one active window → nested requests overwrite body coverage → arithmetic on similarly named clock calls cannot supply joint intervals. Required mechanism: select a valid primitive/protocol and preserve units, invocation identity, calibration and perturbation limits. *Implemented:* explicit replay planning and a read-only SDK timer adapter with work-item0 staging/publication. *Verified:* one-program controls reproduce overwrite; final adapter captures retain ordered joint endpoints for both operators, with measured softmax perturbation. *Proposed:* production backend integration and portable accuracy guarantees; replays remain different executions and official GCU timer frequency is unknown.

[TODO: Turn these verified research-probe mechanisms into the compiler–profiler system in Section 4, then evaluate generality, accuracy and overhead. Unperturbed engine busy time, cross-actor overlap, a measured attribution-error rate, same-SIMT multi-vendor evidence, and operational deployment are not established by this Study.**]**

4 Compiler-Assisted Profiling Design

[TODO: Tie each mechanism to a concrete consequence established in Sections 3.2 and 3.4. Freeze the system boundary before claiming design novelty.**]**

4.1 Profiling Intent and Backend Contracts

[TODO: Define requests, observations, supported capabilities, and the native extension mechanism. Decide between a new profiling IR and extensions to existing IR/metadata. Specify semantics rather than only an API.**]**

4.2 Program Mapping Through Compilation

[TODO: Explain logical-region identity, transformation provenance, one-to-many and many-to-one relationships, and attribution after fusion and scheduling.**]**

4.3 Execution-Aware Instrumentation

[TODO: Describe instrumentation placement and lowering for each execution model. Distinguish issue, completion, and waits; state unsupported quantities. Explain how instrumentation interacts with optimization and synchronization.**]**

4.4 Collection and Reconstruction

[TODO: Define timestamp calibration, buffering, event association, lost-record handling, concurrency, and the separation of measurements from static or derived relationships. Preserve backend-specific observations.**]**

[**TODO:** Draw the final compiler–runtime–analysis architecture, marking inherited components and the profiling-specific extensions introduced by this work.]

Figure 9: System architecture placeholder; mechanisms and boundaries remain to be determined.

5 Implementation

[**TODO:** Record pinned FlagTree/FlagPrism commits and backend toolchains. Document compiler passes, instrumentation runtime, vendor collectors/importers, and profile outputs. Quantify shared versus backend-specific implementation without treating line count as proof of abstraction quality.]

5.1 Backend Integration and Capability Status

[**TODO:** Use a per-chip, per-SDK acceptance matrix. Distinguish an API skeleton, a real capture, successful program association, and fidelity validation. A readable report or passing smoke test does not establish full support.]

5.2 Limitations and Failure Handling

[**TODO:** Specify unavailable clocks/counters, permissions, replay constraints, unsupported IR transformations, ambiguous matches, and explicit degradation.]

6 System Evaluation

The study in Section 3 characterizes the problem. This section will evaluate the proposed mechanisms on separate or held-out tasks and additional backends beyond NVIDIA, Iluvatar, and Ascend. [**TODO:** Freeze study/evaluation separation and avoid selecting evaluation cases only after seeing favorable study results.]

6.1 Measurement and Attribution Fidelity

[**TODO:** Validate event coverage, region association, timestamp accuracy, and reconstruction against independent reference observations and controlled microbenchmarks. State when a reference is not independent.]

6.2 Overhead and Execution Perturbation

[**TODO:** Report per-mode and per-device latency, memory/register use, trace volume, processing cost, and changes to compiled schedules. Check bottleneck/ranking changes caused by instrumentation.]

6.3 Profiling-Guided Operator Optimization

[**TODO:** For each case record the initial hypothesis, additional observation, intervention, code change, correctness check, and profiler-disabled performance. Compare against native tools and reasonable target-local implementations.]

6.4 Mechanism Ablations and Backend Extension

[**TODO:** Remove transformation provenance, execution-aware placement, or native observation detail separately. Show what information or correctness is lost. Measure integration changes for a backend added after common analyses are frozen.]

6.5 Additional Backends

[**TODO:** Validate actual supported modes on Moore Threads, Enflame, MetaX, and other available devices. Report per-backend fidelity, overhead, native detail, and limitations rather than extending the three-backend Study.]

7 Operational Experience and Lessons

[**TODO:** Provide actual adoption context, duration, operators, developer workflows, profiling sessions, resolved/unresolved issues, maintenance burden, and examples of failed hypotheses. The project has not yet supplied bulk usage evidence.]

7.1 Lessons Across Similar and Different Execution Models

[**TODO:** Derive lessons from cases rather than restating architecture descriptions. Discuss which mechanisms were reusable and where the abstraction broke.]

7.2 Scope and Threats to Validity

[**TODO:** Discuss device coverage, compiler/SDK confounding, finite search spaces, measurement perturbation, missing references, and application generalization.]

8 Related Work

Compiler and instruction-level profiling. KPerfIR [2], Neutrino [3], and Proton [10] provide compiler, probe, and multi-level profiling foundations. [**TODO:** Compare concrete supported mechanisms and versions; do not attribute common-counter-only limitations to these systems.]

Context and accelerator analysis. DeepContext [9] connects program contexts across the deep learning stack, and PASTA [6] supplies modular accelerator analysis. **[TODO:** Complete comparisons with HPCToolkit, XProf, RL-Scope, Hotline, Snoopie, DrGPUM, and TenProf using the existing-works survey and primary papers. Separate broader workload diagnosis from our operator profiling scope.]

9 Conclusion

[TODO: Summarize established study findings, implemented mechanisms, measured benefits, and limitations after the empirical work is complete.]

References

[1] FlagOS. FlagTree: A unified compiler for multiple ai chip backends. Project repository. Accessed October 12, 2026. https://github.com/flagos-ai/FlagTree.

[2] Yue Guan, Yuanwei Fang, Keren Zhou, Corbin Robeck, Manman Ren, Zhongkai Yu, Yufei Ding, and Adnan Aziz. KPerfIR: Towards a open and compiler-centric ecosystem for GPU kernel performance tooling on modern AI workloads. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 205–220. USENIX Association, 2025. https://www.usenix.org/conference/osdi25/presentation/guan.

[3] Songlin Huang and Chenshu Wu. Neutrino: Fine-grained GPU kernel profiling via programmable probing. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, pages 331–355. USENIX Association, 2025. https://www.usenix.org/conference/osdi25/presentation/huang-songlin.

[4] Huawei. AI Core architecture. Ascend CANN documentation. Accessed October 12, 2026. https://www.hiascend.com/doc_center/source/en/canncommercial/800/opdevg/tbeaicpudevg/atlasopdev_10_0008.html.

[5] Huawei. CANN 9.0 Ascend C: GetSystemCycle. https://www.hiascend.com/doc_center/source/en/CANNCommunityEdition/900/API/ascendcopapi/atlasascendc_api_07_0282.html. Accessed October 13, 2026.

[6] Mao Lin, Hyeran Jeon, and Keren Zhou. PASTA: A modular program analysis tool framework for accelerators. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. https://2026.cgo.org/details/cgo-2026-papers/28/PASTA-A-Modular-Program-Analysis-Tool-Framework-for-Accelerators.

[7] Moore Threads. MUSA hardware architecture. MUSA SDK 4.3.x programming guide. Historical version; architecture statements apply to its documented devices. Accessed October 12, 2026. https://docs.mthreads.com/musa-sdk/version-4.3.x/programming_guide/Chapter02/.

[8] NVIDIA. Hopper tuning guide. CUDA documentation. Accessed October 12, 2026. https://docs.nvidia.com/cuda/archive/13.2.0/hopper-tuning-guide/index.html.

[9] Qidong Zhao, Hao Wu, Yueming Hao, Zilingfeng Ye, Jiajia Li, Xu Liu, and Keren Zhou. Deepcontext: A context-aware, cross-platform, and cross-framework tool for performance profiling and analysis of deep learning workloads. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 48–63, 2025. https://arxiv.org/abs/2411.02797.

[10] Keren Zhou, Tianle Zhong, Hao Wu, Jihyeong Lee, Yue Guan, Yufei Ding, Corbin Robeck, Yuanwei Fang, Jeff Niu, and Philippe Tillet. Proton: Towards multi-level, adaptive profiling for triton. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2026. https://2026.cgo.org/details/cgo-2026-papers/23/Proton-Towards-Multi-level-Adaptive-Profiling-for-Triton.